CS673 Software Engineering
Team 2 - HealthNest
Project Proposal and Planning

Your project Logo here if any

Team Member	Role(s)	Signature	Date
Deven Shah	Team Lead, Requirements Lead	*Deven Shah*	5/13/2026
Stephanie Wang	QA Lead, Design & Implementation Lead	*Stephanie Wang*	5/13/2026
Aaron Brown	Security Lead, Design & Implementation Lead	*Aaron Brown*	5/12/2026
Chamarr Auber	Team Lead, Configuration Lead	*Chamarr Auber*	5/12/2026

Revision history

Version	**Author**	**Date**	**Change**
1.0	Chamarr Auber, Aaron Brown, Deven Shah, Stephanie Wang	5/13/2026	Initialized all sections of this document.
2.0	Stephanie Wang	5/24/2026	Updated QA section based on feedback and current progress.
3.0	Aaron Brown and Deven Shah	5/29/2026	Update functional requirements and timeline.

1. Overview

Our project is a unified AI-powered medical dashboard for both patients and providers. The idea comes from a common problem in healthcare, where important information is often spread across multiple systems, making it difficult for patients to understand medical instructions, billing details, and follow-up actions, while also creating extra coordination work for providers and care teams.

The goal of the system is to bring important healthcare information and services into one place. At a high level, the system is intended to provide customized home dashboards, AI-assisted support, appointment scheduling, secure access to sensitive information, and one-click access to major services from the dashboard. The main users of the system are patients, doctors, and other care team members.

The planned technology stack currently includes a React frontend, a FastAPI backend, along with database support, authentication, AI integration, testing/security tools, and CI/CD support.

2. Related Work

Some healthcare platforms already provide patients and providers with access to records, appointments, messaging, and other medical information. Two examples are MyChart and InteliChart. MyChart makes it easier to access records, test results, appointments, and other centralized health information. InteliChart is also strong in patient communication and helps improve workflow and coordination.

Both systems are useful, but they still have some limitations. In MyChart, medical information can still be hard to interpret, and the navigation and support experience can feel frustrating. InteliChart also has some usability issues, especially with setup, customization, document access, and messaging.

Our project builds on these kinds of systems by bringing healthcare information, AI assistance, provide workflow support, and scheduling into one platform. The goal is not just to centralize information, but also to make it clearer and more useful for both patients and providers.

3. Proposed High level Requirements

a. Functional Requirements

i. Essential Features

Req ID	Title	Description	Estimated Hours
FR-1	Patient Home Dashboard	As a patient, I want to view my portfolio information and outstanding items in a single unified home dashboard, so that the medical information I need most often is one click away.	20
FR-2	Doctor Home Dashboard	As a doctor, I want to view today's schedule, my inbox (messages, refill requests, results to review), unsigned encounters with age and overdue flags, and a patient quick-lookup field on a single home page, so that all the patient and workload information I need is one click away.	20
FR-3	Patient-Facing AI Assistant (PFA)	As a patient, I want an LLM that knows my records, billing, insurance, and care-team directives via retrieval-augmented generation (no fine-tuning on PHI), so that responses are personalized and auditable without exposing PHI to a non-BAA model provider.	20
FR-4	Doctor-Facing AI Assistant (DFA)	As a doctor, I want a voice-enabled assistant grounded on my schedule, panel, and the active patient's chart, so that I can request information hands-free during clinical work without the model being fine-tuned on patient data.	20
FR-5	Appointment Scheduler	As a system, I want both assistants to have knowledge of both parties' personal schedules and be able to book appointments accordingly, so that they will not have to coordinate directly with each other.	15

ii. Desirable Features

Req ID	Title	Description	Estimated Hours
FR-7	Secure Messaging	As any user, I want to send end-to-end encrypted, audited messages between patients and care teams with attachment support, so that asynchronous communication happens inside the HIPAA boundary.	10
FR-8	PFA Escalation Skill	As a patient, I want the assistant to detect emergencies, suicidal ideation, or out-of-scope questions and route me to 911, a crisis line, or a human staff member, so that I am never left relying on an AI for urgent care.	5
FR-9	PFA Query Skill	As a patient, I want to ask the assistant questions about my medical records, bills, appointments, and prescriptions in text or voice, so that I don't have to navigate the site to find answers.	5
FR-10	PFA Summarizer Skill	As a patient, I want to see doctor directives and clinical notes translated into plain-language summaries with risks and next steps, so that I can act on my care plan without decoding medical jargon.	5
FR-11	PFA Appointment Scheduler Skill		5
FR-12	DFA Visit Overview Skill	As a doctor, I want the assistant to automatically generate a pre-visit summary (recent history, active problems, meds, last labs, open issues) tied to my schedule, so that I am caught up the moment a visit starts without manually prompting.	5
FR-13	DFA Scribe Skill	As a doctor, I want the assistant to optionally transcribe the visit and draft an encounter note for my review and signature, so that documentation does not eat into my visit or after-hours time.	5
FR-14	DFA Query Skill	As a doctor, I want to ask the assistant questions about a patient’s chart,	5

Req ID	Title	Description	Estimated Hours
		medications, labs, visit history, diagnoses, and care gaps using text or voice, so that I can retrieve critical clinical information quickly during patient care without manually navigating the EHR.	
FR-15	DFA Source Attribution Skill	As a doctor, I want AI-generated responses to include references to the underlying chart notes, labs, or records used to generate the answer, so that I can verify clinical accuracy and trust the system output.	5
FR-16	DFA Lab Upload Skill	As a doctor, I want to be able to upload patient lab results directly to Pulse AI, so that I can achieve all my actions and directives through a single interface.	5
FR-18	DFA Appointment Scheduler Skill		5
FR-17	Patient Dashboard - Finance	As a patient, I want to view my outstanding bills and insurance claims, so that any critical finance information or action items are zero clicks away.	15
FR-18	Patient Dashboard - Action Items	As a patient, I want to view my prioritized action-item list, so that any critical finance information or action items are zero clicks away.	5
FR-19	Biometric Authentication	As a user, I want to be able to choose an additional biometric authentication (face, finger, etc.), so that my data is further protected.	5
FR-20	E-Prescribing and Refill Workflow	As a doctor, I want to send and refill prescriptions including controlled substances (EPCS) from the patient chart, so that prescribing happens in-platform.	15

b. Nonfunctional Requirements

Req ID	Title	Description
NFR-1	Authentication	The system will require users to authenticate themselves before providing access to resources.
NFR-2	Authorization (RBAC)	The system will authorize access to resources only after a user has been authenticated. It will also provide role based access control.
NFR-3	Least Privilege	The system will enforce the principle of least privilege and will deny access to resources by default.
NFR-4	Encryption at rest	All sensitive information such as passwords, PII, and PHI will be encrypted before being stored in the database.
NFR-5	Encryption in transit	The system will only use secure message protocols like HTTPS and TLS.
NFR-6	Audit Logging	The system will provide secure and thorough logging of pertinent information, such as login attempts, uploads, access attempts and record modifications, ensuring no sensitive information is exposed.
NFR-7	Password Security	The system will ensure password security through a strong password policy, password expiration, and encryption with salted hashing.
NFR-8	MFA	The system will require multi-factor authentication.
NFR-9	Secure file uploads	The system will restrict file types and sizes for uploads, scan for malware, and prevent executable uploads.
NFR-10	Input validation/sanitization	The system will enforce server-side input validation and sanitization, rejecting malformed or unauthorized inputs and mitigating the risk of executable malicious content.
NFR-11	Session Management	The system will implement secure session management through the use of JWT-based authentication and session timeout policies.
NFR-12	Brute-force protection	The system will implement rate limiting to prevent brute force authentication attacks.
NFR-13	Backup/Disaster Recovery	The system will maintain regular encrypted backups and implement disaster recovery procedures to restore system functionality and data in the event of a system failure, data

Req ID	Title	Description
		corruption or cyberattack.
NFR-14	Data Integrity	The system will ensure data integrity and only authorize the modification of protected resources for authenticated users.
NFR-15	Secrets Management	The system will ensure that secret keys never get exposed.
NFR-16	Dependency Scanning	The system will only use HIPAA approved dependencies, performing routine scans to ensure compliance.
NFR-17	CORS Policy	The system will implement a cross origin resource sharing policy to prevent attacks such cross site request forgery (CSRF)
NFR-18	OWASP Top 10 Mitigation	The system will be built with OWASP top 10 in mind, attempting to mitigate each security risk.
NFR-19	HIPAA-Aware Architecture	The system will attempt to achieve HIPAA compliance.
NFR-20	Mobile Compatibility	The system must display all user interfaces cleanly on both a desktop and mobile browser, including all AI and dashboard features.

4. Management Plan

a. Objectives and Priorities

Below is a list of our project objectives ordered from highest priority to lowest priority:

1) Complete all essential features: All features committed in Iteration 0 are at the highest priority. Completion of these features is synonymous with the completion of our minimum viable product.
2) Ensure all security requirements have been met: Since the platform is dealing with sensitive data with patients and doctors, it is essential that all data is secure at rest and in transit, complying with HIPAA where applicable.
3) Create a collaborative environment where each team member is able to learn and contribute equally to the project throughout the semester: Each team member comes with a unique set of skills, and it is imperative that all work is balanced and allocated accordingly. To accomplish this, responsibilities are clearly identified in the team roles, with active communication in our channels.

4) Ensure there are no defects in the software: High availability and minimal bugs would deliver a smooth and robust platform to our users. Minor bugs and unintended functionalities may be the reason a user is swayed away from using our product.
5) Ensure that the software passes all test cases: Not only for catching the software defects, but ensuring the software passes all test cases would also imply that all functional requirements are met.
6) Design user interfaces that work well on both desktop and mobile devices: Both patients and doctors alike would find themselves using our platform on a laptop, or on a mobile device. We cannot sacrifice UI design quality due to the screen size the user is using. React works well with cross platform compatibility, but having a unique mobile view is a possibility in future iterations.
7) Complete all desirable features: The desirable functional requirements relate directly with the product vision, but will not delay the deployment date if left unfinished. These are features that would be prioritized in the release after the minimum viable product.
8) Complete all optional features: These features may not tie directly to the product vision, but would benefit our users or expand our use cases. Similar to desirable features, unfinished optional features will not delay the deployment date.

b. Risk Management (need to be updated constantly)

The main risks we identified have to do with project management and communication. With a project of this size, it will be hard to keep track of all of the functionalities and requirements during development. We need to have clear systems and processes in place to ensure that each task will be completed within a strict timeline while meeting high quality standards. To help us mitigate these risks, we are going to use discord for communication, Jira for project management and follow the code review guidelines and Github branching strategies we've outlined below.

Here is our **Risk Management Sheet.**

c. Timeline

Iteration	Functional Requirements(Essential/ Disable/Option)	Tasks (Cross requirements tasks)	Estimated /real total person hours

1	- Patient Dashboard - Doctor Dashboard - Appointment Scheduler - Role-Based Access Control	- Set up project structure - Create shared layout - Build basic backend structure	65
2	- Patient-Facing AI Assistant (PFA) - Doctor-Facing AI Assistant (PFA) - PFA Query Skill - DFA Query Skill - Biometric Authentication - Secure Messaging	- Set up AI integration - Build messaging workflow	55
3	- PFA Summarizer Skill - PFA Escalation Skill - PFA Appointment Scheduling Skill - DFA Visit Overview Skill - DFA Scribe Skill - DFA Source Attribution Skill - DFA Lab Upload Skill - DFA Appointment Scheduling Skill - E-Prescribing and Refill workflow	- Improve summary quality - Run final testing - Refine AI assistant	60

5. Configuration Management Plan

a. Tools

Our team will use GitHub to manage our code, documents, and project changes. It will help us keep track of different versions of the project and make it easier for team members to work on different parts without losing other's work. For our IDE we will be using VsCode and containerization will be with docker.
For development, our proposed tech stack includes React with TypeScript for the frontend, FastAPI for the backend, and Supabase for the database. For authentication, we plan to use JWT/OAuth. For AI integrations, we plan to use OpenAI or Claude to support the AI assistant features.

For CI/CD, we will use GitHub Actions. For testing and security tools, we may use SonarQube, ZAP, and Snyk to help with code quality and security checks. For deployment, possible tools include Vercel, AWS, or Google Cloud, depending on what works best for the project. All tasks for this project will be tracked using Jira via a ticket system that all developers will use.

b. Code Commit Guideline and Git Branching Strategy

The code criteria and procedure will be broken down as follows:

Jira: A ticket will be assigned to a developer and that task will have a ticket number

Feature: The first step is the feature branch. When a developer has to add a new feature or fix a bug they must create a feature branch first and title it with the name of the jira ticket first. Ex. “feature/Jira-XXXX-[name of the branch]”. Once the developer creates that branch they can pull from the main branch to get the most updated code. The developer will make their changes and will commit and push the feature branch.
Main: The developer will raise a PR to merge the feature branch to the main branch and the PR must be reviewed by at least one other developer before the author can merge the branch.

Release: We will then create a release branch to push the new feature or bug fix to the production code which will be deployed on vercel/AWS/Google Cloud. The naming convention for the release branch will be “release/release-X.XX”. The developer will then merge the main branch to the release branch.

Production: Once we are ready to push the release to production, we will approve and ship the release branch and host it.

c. CI/CD Plan

All developers will be able to continuously integrate and deploy their code. The code commit and branch strategy I described is used for seamless integration between multiple developers. It ensures that no one's code is being overwritten, it ensures everyone is on the same page because of the naming conventions so everyone knows what each developer is working on. There will be a decrease in bugs being pushed to the main and production branch because all code that is pushed and raised as a PR will go through github actions automated testing to ensure that the code is safe, built cleanly, covered, and bug free before its added. If there is a bug then the developer can see what's wrong immediately and make those

changes without affecting other developers or the working branches. Github actions (the yml files) will be the main focal point as it will be the guard that will evaluate all code that is being added.

6. Quality Assurance Plan

a. Metrics

Metric Name	Description
Essential Features Completed	Number of core features completed compared to what we planned for the iteration.
Effort	Number of actual person hours allocated to a story compared to the estimate.
Velocity	The number of story points completed in an iteration.
Number of Test Cases	Number of test cases created for the features completed in the iteration.
Test Pass Rate	Percentage of test cases that passed in the test run
Open Defects	Number of known issues that were still existed at the end of the iteration
Summary Cases	Number of summary example reviewed to check whether the output is clear and easy to understand
Build/CI Status	Number of pull requests that pass GitHub Actions checks before being merged

b. Coding Standard

Our team will follow a coding standard to keep the project organized, readable, and easier to review. Since the project currently uses a React frontend and a FastAPI backend, we want the code to have clear names, consistent formatting, and a simple structure.
For the frontend, React component names should clearly describe what the component does. Files such as Login.jsx, PatientDashboard.jsx, and DoctorDashboard.jsx should stay focused on their own feature area. JavaScript and JSX files should follow the project's ESLint rules, and code should be formatted before it is committed.

For the backend, FastAPI files should continue to be organized by responsibility. Route files should handle API endpoints and request routing, service files should handle business logic, validation, and response formatting, schema files should define request and response structures, and the Supabase client should stay separate from route logic. This matches the current authentication backend structure and should continue as more backend features are added.
Comments should be added when the logic is not obvious, and repeated code should be avoided when possible. Any user input should be validated before it is used, especially for authentication. As dashboard data, lab result uploads, scheduling, and messaging are added, the same validation and organization standards should be followed.

c. Code Review Process

The person who writes the code should first make sure the feature runs properly on their local machine and matches the Jira task or feature they were working on. Before opening a pull request, the developer should also check that the code is readable, formatted consistently, and does not break the existing project.
After a pull request, the Configuration Lead will help make sure the review and merge process is being followed. It is the Configuration Lead's responsibility to ensure every pull request is assigned a reviewer (one who has not contributed directly to the code).
The QA Lead will review the pull request from a testing perspective. This includes looking for possible bugs, missing validation, unclear user flows, dashboard behavior issues, and anything that could affect integration with other features. The QA Lead should also check whether the feature can be tested using the planned acceptance tests.
The Requirements Lead will review the pull request when needed to make sure the implementation still matches the related functional requirement or user story. If the feature does not meet the requirement, the developer should make the needed changes before the code is merged.

- As part of the review, teammates will use a simple checklist:
- The code runs locally without errors.
- The feature matches the Jira task or requirement.
- The code is clear and easy to follow.
- Formatting is consistent with the rest of the project.
- Repeated code is avoided when possible.
- Helpful comments are included when the logic is not obvious.
- Input validation is included where needed.
- The change does not create obvious testing or integration problems.

d. Testing

Our team will use a combination of developer testing, QA review, manual acceptance testing, CI/CD checks, and automated testing to make sure the system works correctly. As the project continues, manual testing will be used for the main user flows, and automated tests will be added as the frontend and backend features become more stable.

Developers should first test the parts they worked on before opening a pull request. This includes running the project locally, checking that the feature works as expected, and making sure it does not break existing functionality. For Iteration 1, this includes testing the login, sign-up, sign-out, and patient/provider dashboard routing flow.
The QA Lead will focus on testing the main workflows from a user perspective. This includes checking whether the patient and provider dashboards behave correctly, whether the authentication flow is understandable, whether validation or error handling is missing, and whether the feature matches the planned acceptance tests. Manual testing will also be used for healthcare-specific flows such as lab result review, appointment scheduling, secure messaging, and AI-assisted summaries as those features are added.
Automated testing will be part of the project plan. Frontend tests should be added for important React components and helper functions, such as authentication behavior, dashboard rendering, and role-based dashboard display. Backend tests should be added for FastAPI routes and service logic, especially authentication and future API endpoints for dashboard data, lab results, scheduling, and messaging. Integration tests should also be added to check connected flows between the frontend, backend, and Supabase.
GitHub Actions will support the testing process by running checks before code is merged. The current CI/CD process includes linting and build checks, and automated test execution should be added once test suites are created. If a GitHub Actions check fails, the pull request should be corrected before it is merged into the main branch.

e. Defect Management

GitHub Issues will be used to track defects since it is already connected to the team repository. The team will mainly look for problems like incorrect dashboard information, scheduling conflicts, unclear summaries, AI-related mistakes, and issues between connected features. When an issue is found, it should be marked with a short description and the feature it affects. The QA Lead will keep track of which issues still require attention, and the teammate who worked on that feature will handle the fix if possible. Issues that affect features like dashboard behavior, summaries, or scheduling should be treated as higher priority and fixed before the end of the iteration.

7. AI usage Log

Section	Who	AI contribution (%)	Description	Verified by
3	Aaron	30%	Used ChatGPT to help decide on non-functional security	Aaron

Section	Who	AI contribution (%)	Description	Verified by
			requirements.	
3	Deven	30%	Used Opus 4.7 to help analyze functional requirements based on current SOTA in the market right now.	Deven
4	Stephanie	10%	Used ChatGPT to help calculate realistic estimated person hours for the timeline based on the listed functional requirement.	Stephanie
6	Stephanie	30%	Used ChatGPT to brainstorm QA plan examples, suggest common software quality metrics, and check grammar/wording.	Stephanie
6	Stephanie	30%	Used ChatGPT to revise the QA section based on feedback and current workflow. AI helped clarify how to describe the current process while keeping the section aligned with the current stage and plan.	Stephanie

8. Project Links

Github
ChatGPT
Claude AI

9. Glossary

BAA: Business Associates Agreement.
CORS: Cross-Origin Resource Sharing.
CSRF: Cross-Site Request Forgery.
DFA: Doctor Facing Assistant.
EPCS: Electronic Prescriptions for Controlled Substances.
HIPAA: Health Insurance Portability and Accountability Act.
HTTPS: Hyper-Text Transfer Protocol Secure.

JWT: JSON Web Tokens.
MFA: Multi-Factor Authentication.
OWASP: Open Worldwide Application Security Project.
PFA: Patient Facing Assistant.
PHI: Personal Health Information.
PII: Personally Identifiable Information.
RBAC: Role-Based Access Control.
TLS: Transport Layer Security.